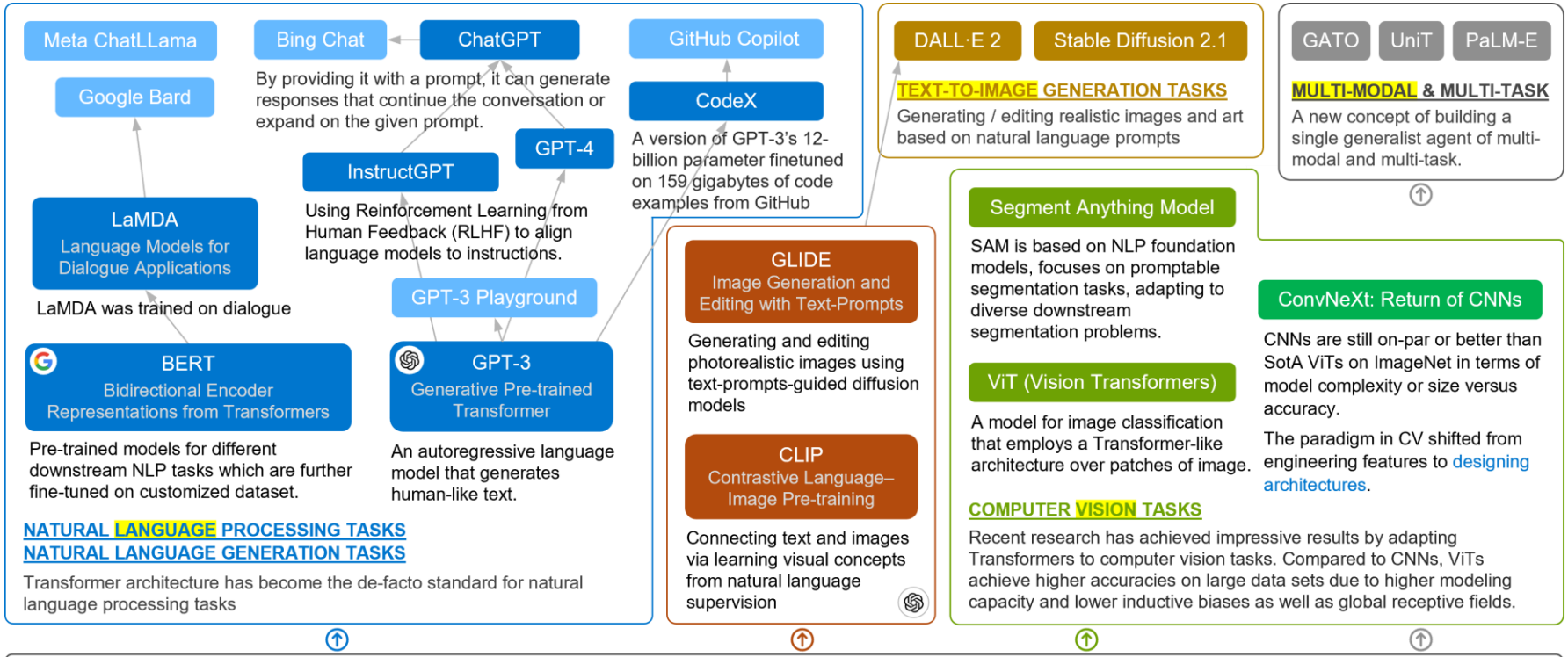
1. Transformer is All You Need
2. Large Language Models
3. Generative AI Techniques

Transformers Take Over Artificial Intelligence



Transformers **Large-scale Transformer models triggered breakthrough in AI-Generated Content**

A Transformer is a type of neural network architecture based on the concept of self-attention, which allows the model to focus on specific parts of the input sentence to better understand the context of the sentence. Transformer models have been used to achieve state-of-the-art results in many NLP tasks, such as machine translation, text summarization, and question answering.

ARTIFICIAL INTELLIGENCE 2023

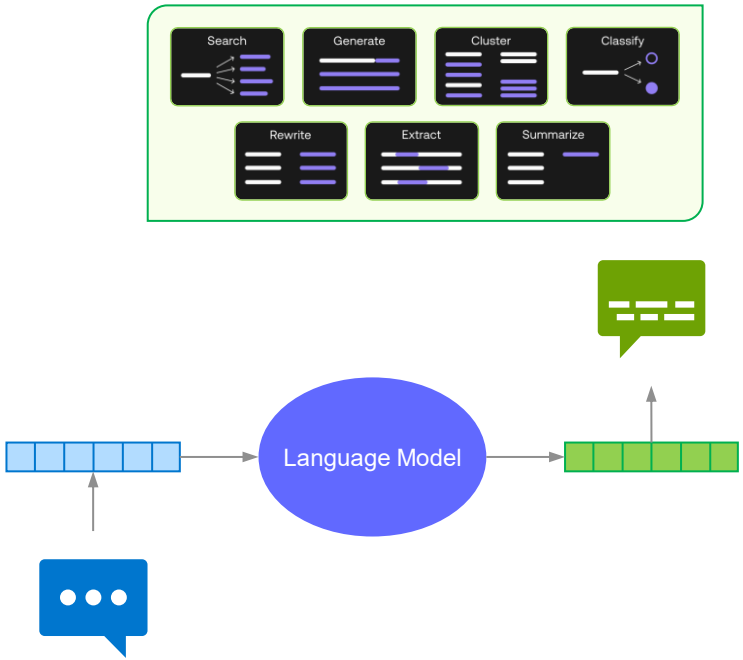
Large Language Models

Sanping Li

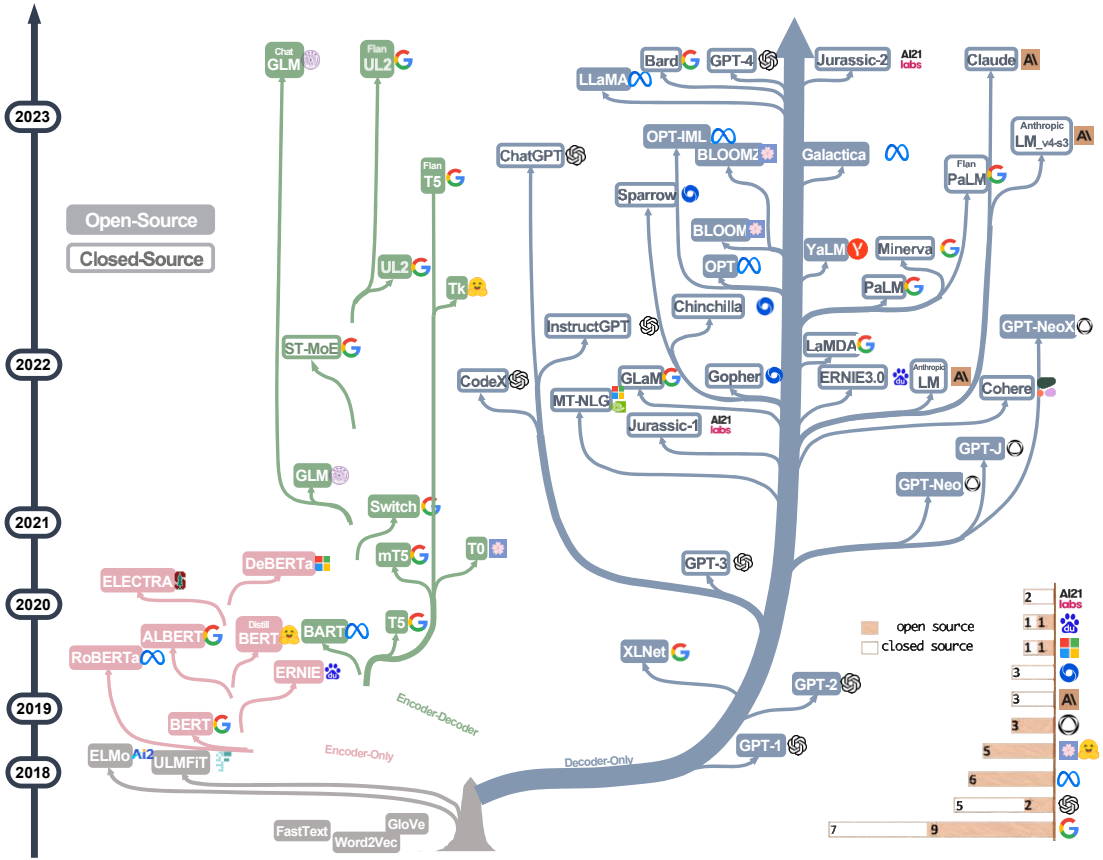
Apr 2023

Technical Terms

- Transformer
  - A deep learning model that adopts the mechanism of self-attention, differentially weighting the significance of each part of the input data. It is used primarily in the fields of natural language processing (NLP) and computer vision (CV). [Wikipedia]
- Language model
  - A representation of probability distribution over words or word sequences.
- Large Language Model (LLM)
  - A type of general-purpose machine learning model that can handle a wide range of NLP use cases.
- Conversational AI
  - A type of AI technology that can simulate human conversation.
- Generative AI
  - A type of AI technology that can produce various types of content including text, imagery, audio and synthetic data.



The Evolutionary Tree of Modern LLMs



https://arxiv.org/abs/2304.13712 https://github.com/Mooler0410/LLMsPracticalGuide

Decoder-only models have been gradually dominating the development of LLMs.

- At the early stage of LLMs development, decoder-only models were not as popular as encoder-only and encoder-decoder models.
- However, after 2021, with the introduction of game-changing LLMs - GPT-3, decoder-only models experienced a significant boom.
- Meanwhile, after the initial explosive growth brought about by BERT, encoder-only models gradually began to fade away.

The evolutionary tree of modern LLMs traces the development of language models in recent years and highlights some of the most well-known models.

- Models on the same branch have closer relationships.
- Transformer-based models are shown in non-grey colors, decoder-only models in the blue branch, encoder-only models in the pink branch, and encoder-decoder models in the green branch.
- The vertical position of the models on the timeline represents their release dates.
- Open-source models are represented by solid squares, while closed-source models are represented by hollow ones.
- The stacked bar plot in the bottom right corner shows the number of models from various companies and institutions.

Summary of Large Language Models

	Characteristic		LLMs
Encoder-Decoder or Encoder-only (BERT-style)	Training: Model type: Pretrain task:	Masked Language Models Discriminative Predict masked words	ELMo [80], BERT [28], RoBERTa [65], DistilBERT [90], BioBERT [57], XLM [54], Xlnet [119], ALBERT [55], ELECTRA [24], T5 [84], GLM [123], XLM-E [20], ST-MoE [133], AlexaTM [95]
Decoder-only (GPT-style)	Training Model type: Pretrain task:	Autoregressive Language Models Generative Predict next word	GPT-3 [16], OPT [126]. PaLM [22], BLOOM [92], MT-NLG [93], GLaM [32],Gopher [83], chinchilla [41], LaMDA [102], GPT-J [107], LLaMA [103], GPT-4 [76], BloombergGPT [117]

- BERT-style Language Models: Encoder-Decoder or Encoder-only
  - As natural language data is readily available and unsupervised training paradigms have been proposed to better utilize extremely large datasets, this motivates the unsupervised learning of natural language.
  - One common approach is to predict masked words in a sentence while considering the surrounding context. This training paradigm is known as the Masked Language Model.
  - This type of training allows the model to develop a deeper understanding of the relationships between words and the context in which they are used.
- GPT-style Language Models: Decoder-only
  - Researchers found that scaling up language models significantly improves the few-shot, even zero-shot performance.
  - The most successful models for better few-shot and zero-show performance are Autoregressive Language Models, which are trained by generating the next word in a sequence given the preceding words.
  - These models have been widely used for downstream tasks such as text generation and question answering.
  - Examples of Autoregressive Language Models include GPT-3, OPT, PaLM, and BLOOM.

https://arxiv.org/pdf/2304.13712.pdf

GPT and More Language Models

Back to Table of Contents

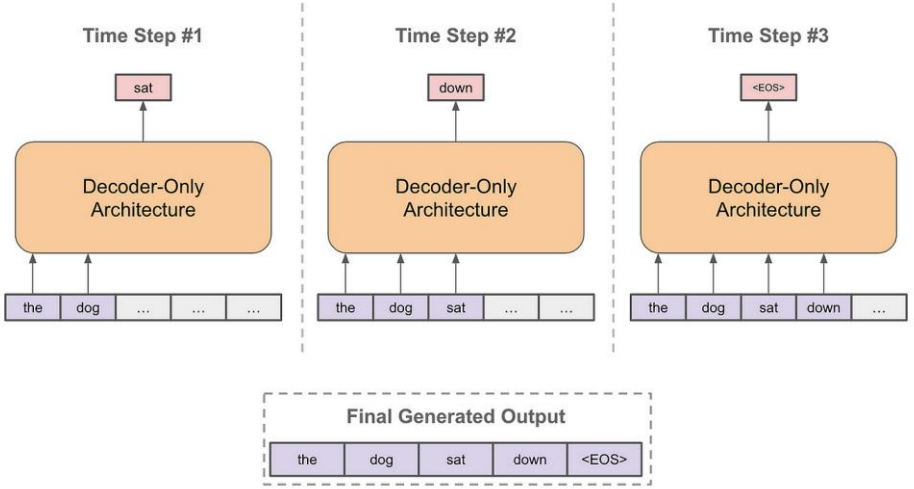
GPT: Generative Pretrained Transformers

- Generative Pretrained Transformers (GPT) are a family of autoregressive language models developed by OpenAI generally trained on a large corpus of text data such that they can generate human-like text.

- An autoregressive language model predicts the next word in a sequence of words based on the words that have come before it.
  - This can be used for tasks such as natural language processing and machine translation.

- An autoregressive model is a type of statistical model that uses past values of a time series to predict future values.
  - It is based on the assumption that the current value of the time series depends on its past values, with the relationship between the current and past values described by a set of coefficients.

To forecast the outcome of the next time step, an autoregressive model uses the results of prior time steps as inputs into a regression model.



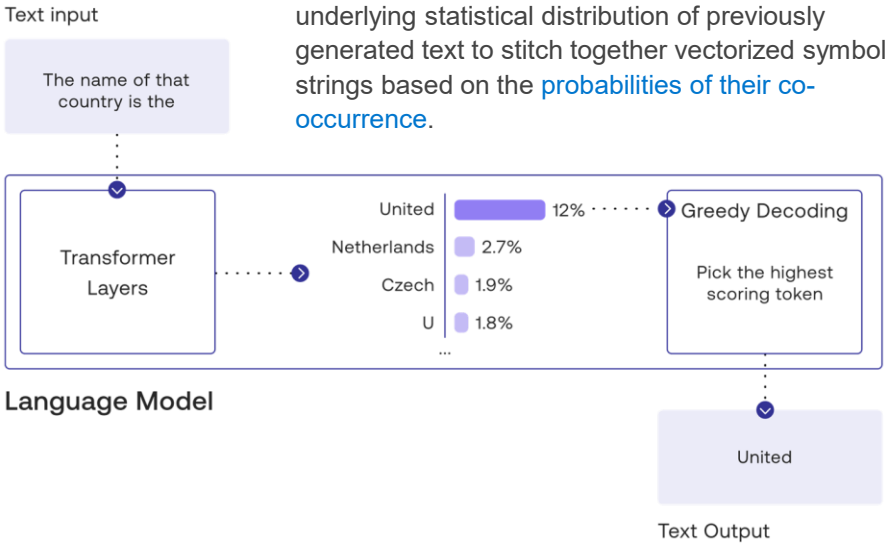
https://deepchecks.com/glossary/autoregressive-model/

Generative Models

- Informally, a generative model is one that can generate new data after learning from the dataset.
- Generative models have been revolutionizing natural language processing by playing a pivotal role in automating tasks like text generation. These models are trained on specialized data sets to predict a probable future output based on the input provided.
- Generative models can accurately capture relationships between words and phrases that traditional techniques cannot detect, thus allowing them to generate more accurate outputs.
- These models can be trained to learn from large datasets without needing human guidance or intervention.

https://azumo.com/insights/a-quick-guide-to-generative-models-and-gpt-3

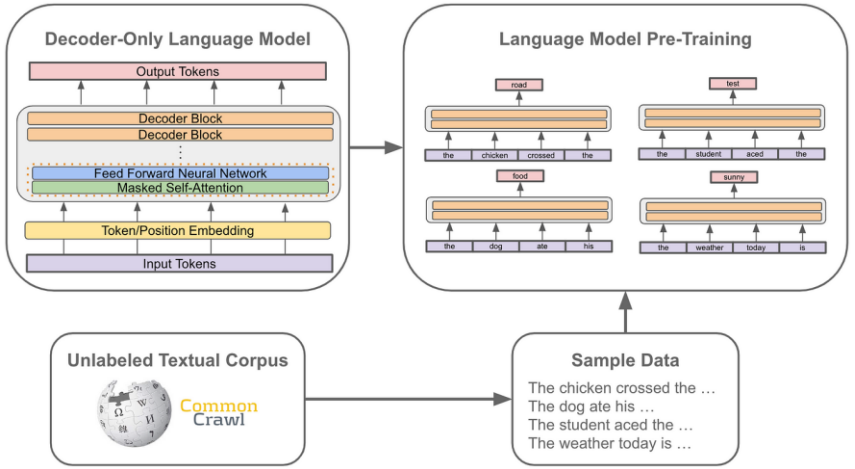
- LLMs generate predictions of the 'statistically likely continuations of word sequences' based on brute-force iterative training on massive corpuses of digital text data.
- As sequence predictors, these models draw on the underlying statistical distribution of previously generated text to stitch together vectorized symbol strings based on the probabilities of their co-occurrence.



https://docs.cohere.ai/docs/controlling-generation-with-top-k-top-p

Pretrained by Self-Supervised Learning

- GPT models are pre-trained over a corpus/dataset of unlabeled textual data using a language modeling objective.
  - Put simply, this means (i) sampling some text from the dataset and (ii) training the model to predict the next word.
  - This pre-training procedure is a form of self-supervised learning, as the correct "next" word can be determined by simply looking at the next word in the dataset.



- Such self-supervised training methods have been particularly successful when used in conjunction with a new architecture for deep neural networks called the transformer.
  - At its most basic level, a transformer is a neural network optimized for processing sequences with long-range dependencies (for example, words far apart in a sentence that depend on one another), using the idea of "attention" weights to focus processing on the most relevant parts of the data.
- Transformers with self-supervised learning are a promising tool for creating more general AI systems
  - Applicable to or easily integrated with a wide variety of data — text, images, even protein-folding structures
  - Once trained, they can either immediately or with just "fine-tuning" achieve state-of-the-art narrow AI performance on difficult tasks.

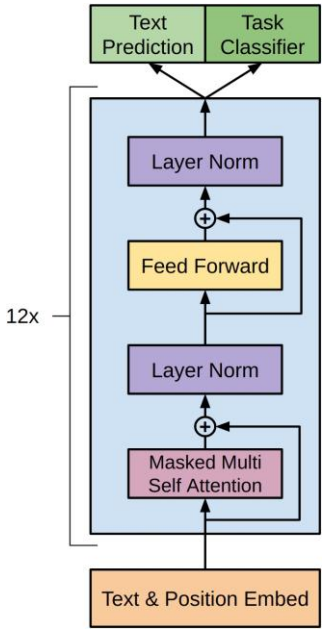
https://ai100.stanford.edu/2021-report/standing-questions-and-responses/sq5-what-are-prospects-more-general-artificial

Transformer Architecture

- The transformer adopts an architecture based on encoder-decoder stacking, with the use of attention mechanism, differentially weighing the significance of each part of the input data.
- Each encoder consists of a Self-Attention layer followed by the Feed Forward network. Self-attention uses trained embeddings from the same layer to compute the attention vector. It passes its encodings to the next encoder layer as inputs.
- The decoder also has the same layers as the encoder, except that additional encoder-decoder attention is introduced in between to help the model extract relevant features from attention vectors from the encoder.
- Both the encoder and decoder layers have a feed-forward neural network for additional processing of the outputs, and contain residual connections and layer normalization steps.

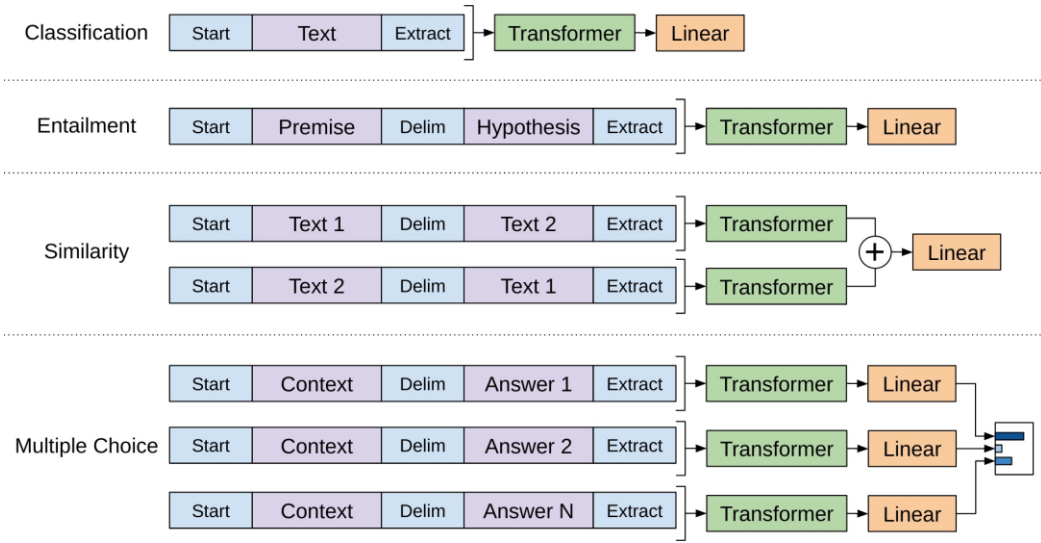
https://en.wikipedia.org/wiki/Transformer_(machine_learning_model) https://www.revistek.com/posts/transformer-architecture

GPT Model Architecture



Input transformations for fine-tuning on different tasks.

All structured inputs are converted into token sequences to be processed by pre-trained model, followed by a linear + softmax layer.



Textual entailment For entailment tasks, we concatenate the premise p and hypothesis h token sequences, with a delimiter token ($) in between.

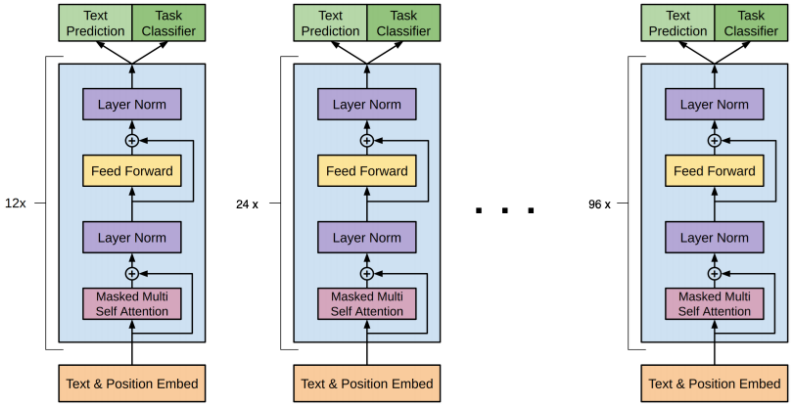
Similarity For similarity tasks, there is no inherent ordering of the two sentences being compared. To reflect this, we modify the input sequence to contain both possible sentence orderings (with a delimiter in between) and process each independently to produce two sequence representations which are added element-wise before being fed into the linear output layer.

Question Answering and Commonsense Reasoning For these tasks, we are given a context document z, a question q, and a set of possible answers {ak}. We concatenate the document context and question with each possible answer, adding a delimiter token in between to get [z; q; $; ak]. Each of these sequences are processed independently with the model.

- 12-layer stacked decoder-only transformer with masked self-attention heads (768 dimensional states and 12 attention heads).
- Each layer of the decoder simply consists of a masked self-attention layer followed by a feed forward neural network. For the position-wise feed-forward networks, we used 3072 dimensional inner states.
- Masked self-attention is required for language modeling because we should not be able to look forward in the sentence while predicting the next token.

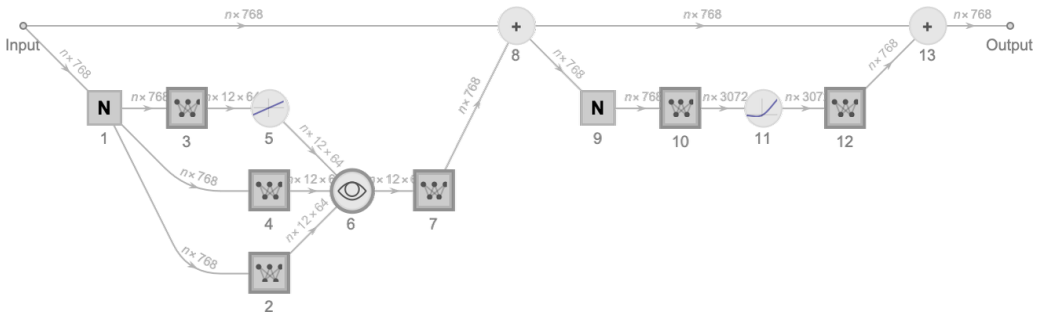
https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf

GPT, GPT-2, GPT-3



Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or "GPT-3"	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

Table 2.1: Sizes, architectures, and learning hyper-parameters (batch size in tokens and learning rate) of the models which we trained. All models were trained for a total of 300 billion tokens.



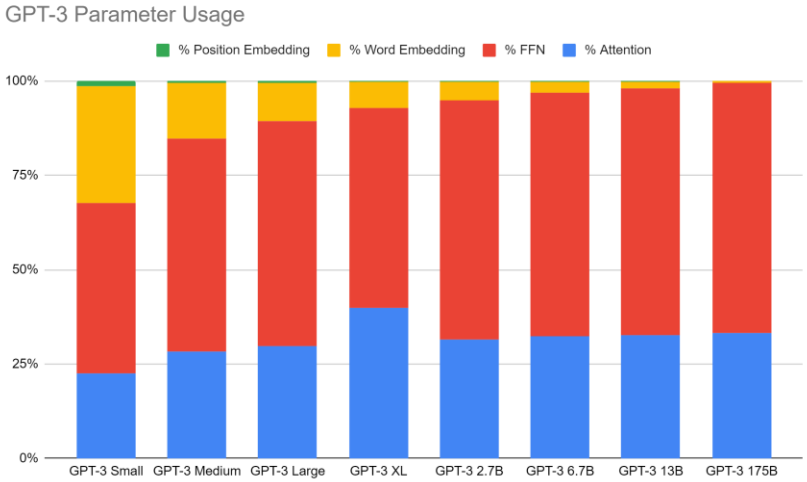
- Table 2.1 shows the sizes and architectures of our 8 models. Here n_{params} is the total number of trainable parameters, n_{layers} is the total number of layers, d_{model} is the number of units in each bottleneck layer (we always have the feedforward layer four times the size of the bottleneck layer, $d_{ff} = 4 * d_{model}$), and d_{head} is the dimension of each attention head. All models use a context window of n_{ctx} = 2048 tokens.
- We partition the model across GPUs along both the depth and width dimension in order to minimize data-transfer between nodes. The precise architectural parameters for each model are chosen based on computational efficiency and load-balancing in the layout of models across GPU's.

https://arxiv.org/pdf/2005.14165.pdf https://writings.stephenwolfram.com/2023/02/what-is-chatgpt-doing-and-why-does-it-work/

How does GPT-3 Spend its 175B Parameters?

Parameter name ->	n_params	n_layers	d_model	n_heads	d_head	n_ctx	n_vocab	Predicted n_params	% Error	% Attention	% FFN	% Word Embedc	% Position Embe	Loose-but	Loose pre
Description ->	Total num	total numt	number of units in e		dimensior	tokens in	vocabulary size								
Abbreviation ->		x	y	z	w	u	v	4xyzw+8xy^2+5xy+vy+uy						12xy^2+vy	
GPT-3 Small	1.25E+08	12	768	12	64	2048	50527	1.25E+08	-0.3%	22.58%	45.21%	30.96%	1.25%	1.24E+08	1.0%
GPT-3 Medium	3.50E+08	24	1024	16	64	2048	50527	3.56E+08	-1.7%	28.28%	56.59%	14.54%	0.59%	3.54E+08	-1.1%
GPT-3 Large	7.60E+08	24	1536	16	96	2048	50527	7.60E+08	-0.1%	29.79%	59.59%	10.21%	0.41%	7.57E+08	0.4%
GPT-3 XL	1.30E+09	24	2048	24	128	2048	50527	1.52E+09	-16.7%	39.81%	53.09%	6.82%	0.28%	1.31E+09	-0.9%
GPT-3 2.7B	2.70E+09	32	2560	32	80	2048	50527	2.65E+09	1.8%	31.64%	63.29%	4.88%	0.20%	2.65E+09	2.0%
GPT-3 6.7B	6.70E+09	32	4096	32	128	2048	50527	6.66E+09	0.6%	32.25%	64.51%	3.11%	0.13%	6.65E+09	0.8%
GPT-3 13B	1.30E+10	40	5140	40	128	2048	50527	1.29E+10	0.5%	32.55%	65.36%	2.01%	0.08%	1.29E+10	0.5%
GPT-3 175B	1.75E+11	96	12288	96	128	2048	50527	1.75E+11	0.2%	33.21%	66.42%	0.36%	0.01%	1.75E+11	0.2%

https://docs.google.com/spreadsheets/d/10Y4GLc28UgeKr2qSYEZuRqELn1D-w5EiQpAGg-_y4Xg/edit#gid=899002403



- For large models, the only terms that contribute significantly to the parameter count are the attention weights and non-bias FFN weights. So parameter cost increases directly with the number of layers, but with the square of the internal dimensionality.
- Position embedding always take very few parameters.
- Word embedding takes about 30% of the parameters for the smallest model, but a proportionally smaller amount as the model gets larger, ultimately <1% of parameters for the full-size GPT-3.
- The remaining parameters are split 2:1 between the feed-forward network and the attention heads, except in GPT-3 XL.

https://www.lesswrong.com/posts/3duR8CrvcHywrnhLo/how-does-gpt-3-spend-its-175b-parameters https://dugas.ch/artificial_curiosity/GPT_architecture.html

GPT-4: Accepting Text and Image Inputs

- *GPT-4 is a large multimodal model (accepting image and text inputs, emitting text outputs).*
- *We've spent 6 months iteratively aligning GPT-4 using lessons from our adversarial testing program as well as ChatGPT, resulting in our best-ever results (though far from perfect) on factuality, steerability, and refusing to go outside of guardrails.*
- *Over the past two years, we rebuilt our entire deep learning stack and, together with Azure, co-designed a supercomputer from the ground up for our workload. A year ago, we trained GPT-3.5 as a first "test run" of the system. We found and fixed some bugs and improved our theoretical foundations. As a result, our GPT-4 training run was (for us at least!) unprecedentedly stable, becoming our first large model whose training performance we were able to accurately predict ahead of time.*

https://albertoromgar.medium.com/the-era-of-large-ai-models-is-over-a5c9d7d804d4

> *But then, why build GPT-4 at all? Why make it larger than anything else if it's so costly to train and run? I don't think GPT-4's existence is incoherent with Altman's belief that the AI community must exploit approaches other than scale. One likely explanation is that GPT-4 wasn't as much motivated by the artificial-human-brain fantasy as by the necessity to build a moat sufficiently deep that just looking at it over the edge would imbue doubts in OpenAI's competitors.*
>
> https://albertoromgar.medium.com/the-era-of-large-ai-models-is-over-a5c9d7d804d4

AI

May 19, 2020

Microsoft announces new supercomputer, lays out vision for future AI work

By Jennifer Langston

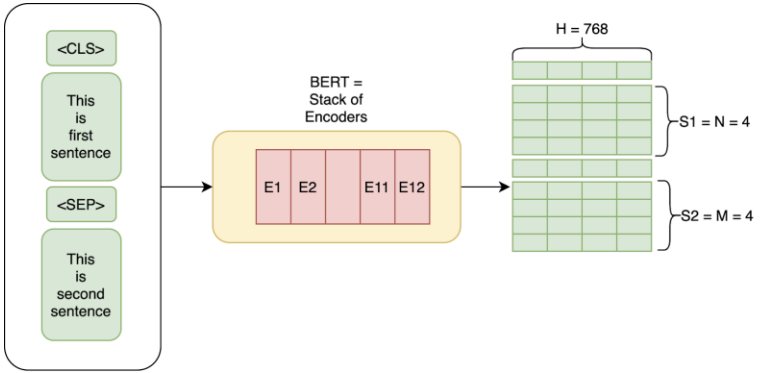
*The supercomputer developed **for OpenAI** is a single system with more than **285,000 CPU cores, 10,000 GPUs and 400 gigabits per second of network connectivity for each GPU server**. Compared with other machines listed on the TOP500 supercomputers in the world, it ranks in the top five, Microsoft says. Hosted in Azure, the supercomputer also benefits from all the capabilities of a robust modern cloud infrastructure, including rapid deployment, sustainable datacenters and access to Azure services.*

https://news.microsoft.com/source/features/ai/openai-azure-supercomputer/

BERT

- Bidirectional Encoder Representations from Transformers (BERT) is a family of masked-language models published in 2018 by researchers at Google.
- BERT is based on the transformer architecture. Specifically, BERT is composed of Transformer encoder layers.
- BERT was pre-trained simultaneously on two tasks: language modeling (15% of tokens were masked, and the training objective was to predict the original token given its context) and next sentence prediction (the training objective was to classify if two spans of text appeared sequentially in the training corpus).
- After pre-training, BERT can be fine-tuned with fewer resources on smaller datasets to optimize its performance on specific tasks such as NLP tasks (language inference, text classification) and sequence-to-sequence based language generation tasks (question-answering, conversational response generation).

https://en.wikipedia.org/wiki/BERT_(language_model)



https://www.exxactcorp.com/blog/Deep-Learning/how-do-bert-transformers-work

Popular Open-Source Alternatives

Model	Organization	# Paras	Size	Architecture	Features	Training	Access
Bloom	BigScience Workshop	176B	330GB	A decoder-only architecture, created based on 8.3B Megatron-LM	Support 46 languages and 13 programming languages	Trained for 3.5 months on 384 A100 80GB GPUs	https://huggingface.co/bigscience/bloom
OPT	Meta	175B	328GB	Autoregressive multilingual LLM	OPT combines both pretrained models and the source code for using.	992 80GB A100 GPUs	https://github.com/facebookresearch/metaseq/tree/main/projects/OPT
BERT	Google	340M		BERT is a bidirectional, unsupervised language representation.	104 languages in multilingual model		https://github.com/google-research/bert
GPT-NeoX-20B	EleutherAI	20B	268GB	Autoregressive language model trained on the Pile.	Same as GPT-3		https://github.com/EleutherAI/gpt-neox
CodeGen	Salesforce	16B		A family of autoregressive language models	For program synthesis, with next-token prediction language modeling as the learning objective.	TPU-v4	https://github.com/salesforce/CodeGen

*** The list of more language models and transformer models can be found online.*

- Even though open-source development is by definition open for contributors to use, it will incur high development costs. Hosting, training and even fine-tuning these models is a further drain, as it requires investment, specialized knowledge and large volumes of specially connected GPUs.
- In other words, investment and human guidance are required for these technologies to be useful for business applications. Often, adaptation of models can be achieved through either fine-tuning on certain amounts of human-labeled data, or continuous interaction with developers and the results they generated from the models.

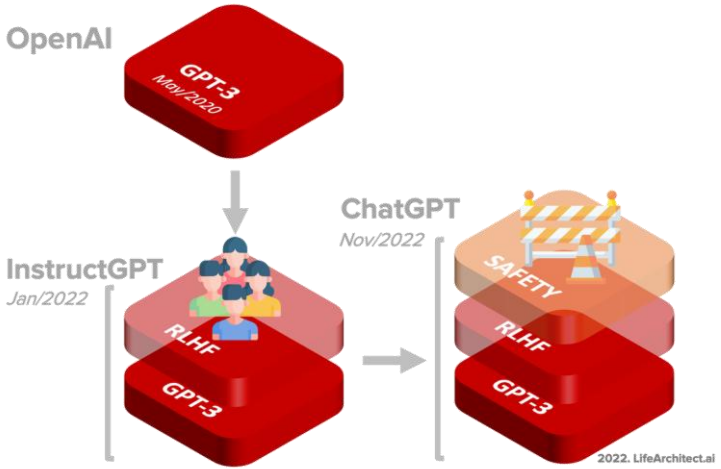
Low-Cost Open-Source Replications of ChatGPT

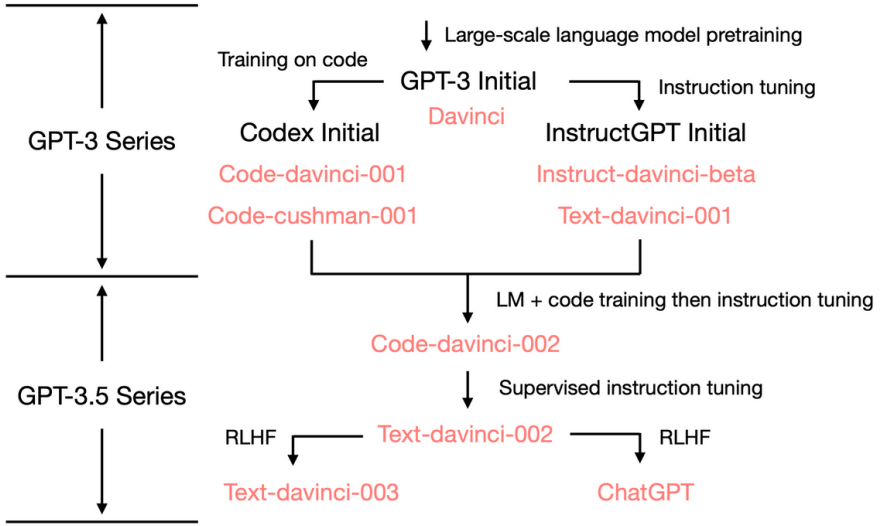
- Since OpenAI has not released the code for ChatGPT, how to effectively replicate ChatGPT has become a huge problem faced by everyone, and an open-source ChatGPT equivalent is in great demand.
  - Although ChatGPT has been released for several months, neither open-source pre-training weight nor a complete open-source training process at low cost is available to the public. In fact, it's difficult to realize the efficient replication of the whole process of ChatGPT-style service based on kinds of 100-billion-parameter models.
- One good option is Colossal-AI, as one of the hottest open-source solutions for large AI models, presents an open-source low-cost ChatGPT equivalent implementation process. [GitHub link] [**ColossalChat**]
  - Colossal-AI greatly reduces the GPU memory overhead of ChatGPT training by using ZeRO, Gemini, LoRA, AutoChunk memory management, etc.
- Alpaca AI: Stanford researchers clone ChatGPT AI for just $600
  - A critical component of this achievement was LLaMA 7B, an open-source language model, which the researchers got access to. They use 175 human-written instruction/output pairs to generate more in the same style and format. Generating 20 such statements at a time, the researchers amassed 52,000 sample conversations in very little time, which cost them $500. This dataset was then used to post-train the LLaMa model.
- Dolly: The release of Dolly is the first in a series of announcements Databricks is making that focus on helping every organization harness the power of large language models.
  - Dolly works by taking an existing open source 6 billion parameter model from EleutherAI and modifying it ever so slightly to elicit instruction following capabilities such as brainstorming and text generation not present in the original model, using data from Alpaca.

Training of Large Language Models

Back to Table of Contents

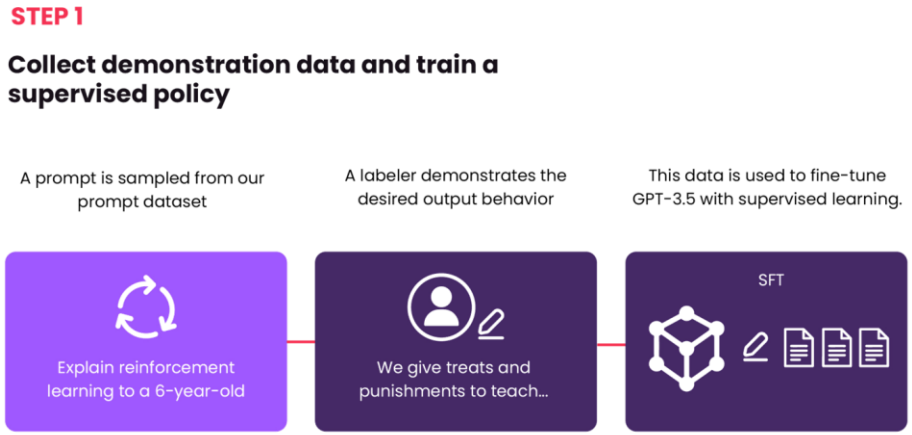
Brief Overview of ChatGPT Training





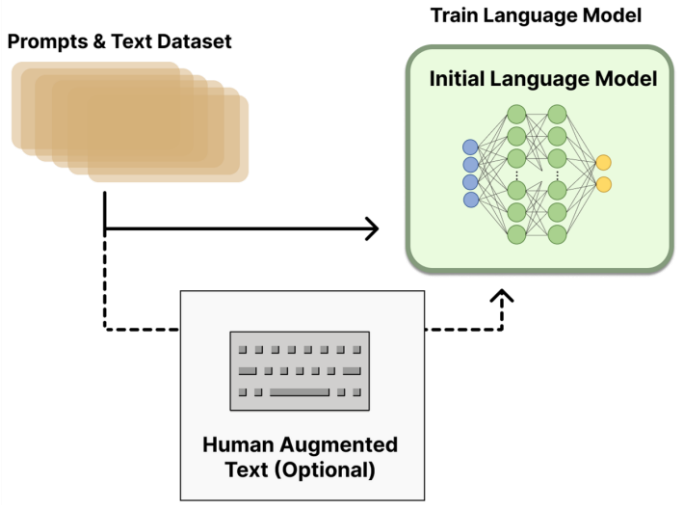
https://xv44586.github.io/2023/01/09/zero-to-chatgpt/

ChatGPT Training: Supervised Fine-Tuning



- Labelers provide demonstrations of the desired behavior on the input prompt distribution. Then fine-tune a pretrained GPT-3 model on this data using supervised learning.
- Supervised Fine-Tuning (SFT): the model is trained for 16 epochs, using a cosine learning rate decay, and residual dropout of 0.2.

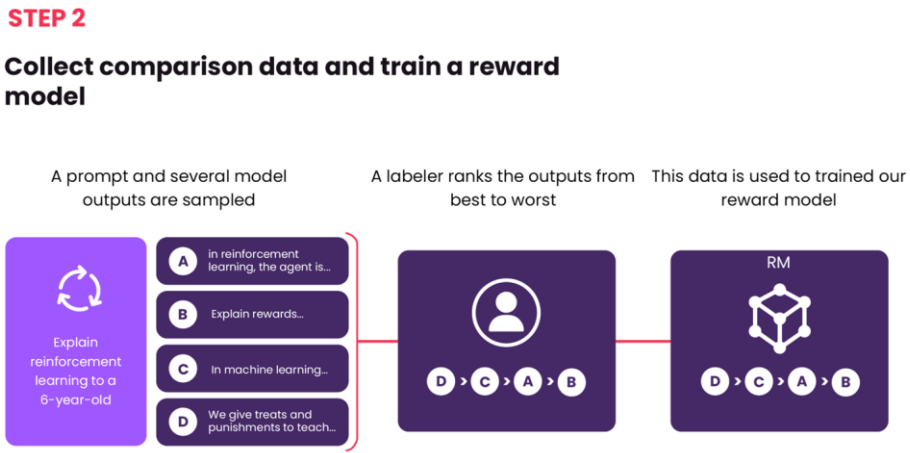
https://www.aivo.co/blog/chatgpt-training-process-advantages-and-limitations

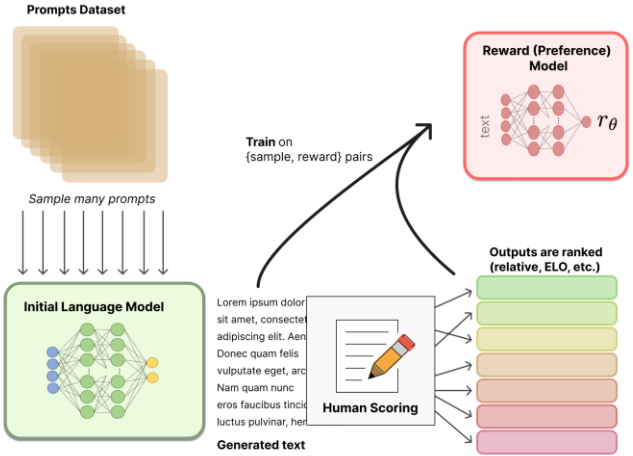


https://huggingface.co/blog/rlhf

- This initial model can also be fine-tuned on additional text or conditions, but does not necessarily need to be.
- For example, OpenAI fine-tuned on human-generated text that was “preferable” and Anthropic generated their initial LM for RLHF by distilling an original LM on context clues for their “helpful, honest, and harmless” criteria.

ChatGPT Training: Human Feedback





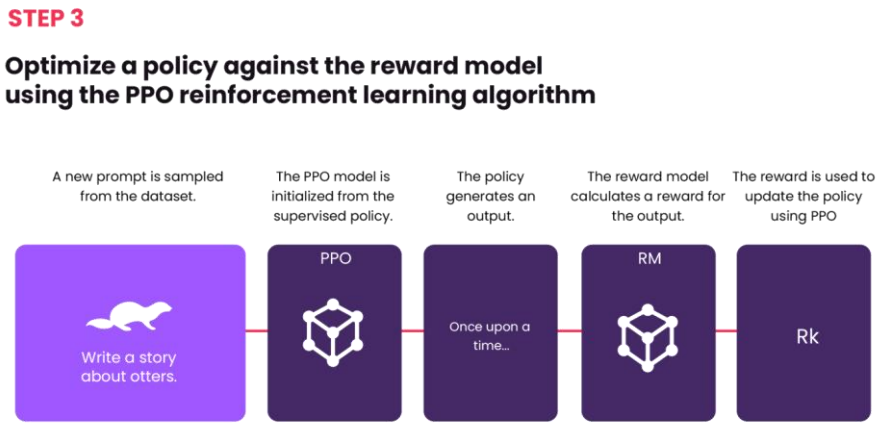
In order to speed up comparison collection, we present labelers with anywhere between $K = 4$ and $K = 9$ responses to rank. This produces $\binom{K}{2}$ comparisons for each prompt shown to a labeler.

- The second step is the most interesting, since different responses are proposed to the model and human feedback provides a ranking of responses from most desirable to least desirable. Using this information, a model can be trained, which is "rewarded" and captures human preferences, reducing misalignment.

https://www.aivo.co/blog/chatgpt-training-process-advantages-and-limitations

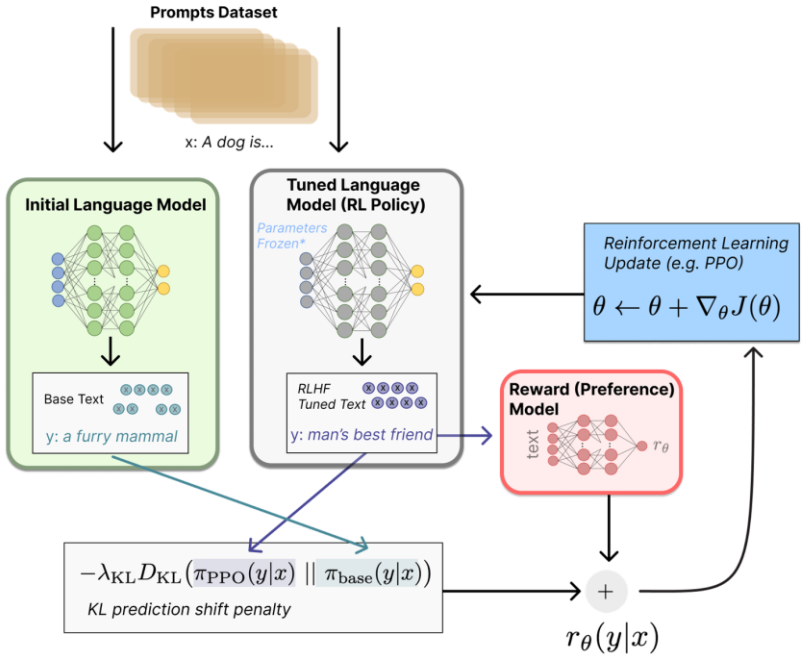
- Collect a dataset of comparisons between model outputs, where labelers indicate which output they prefer for a given input. Then train a reward model to predict the human-preferred output.
- Reward modeling (RM). Starting from the SFT model with the final unembedding layer removed, we trained a model to take in a prompt and response and output a scalar reward.
- In this paper we only use 6B RMs, as this saves a lot of compute, and we found that 175B RM training could be unstable and thus was less suitable to be used as the value function during RL.

ChatGPT Training: Reinforcement Learning



- Step 3 is to treat GPT as a “policy” (Reinforcement Learning term) and optimize it using RL against the learned reward.
- An algorithm called PPO is used and, in this way, GPT is better aligned.

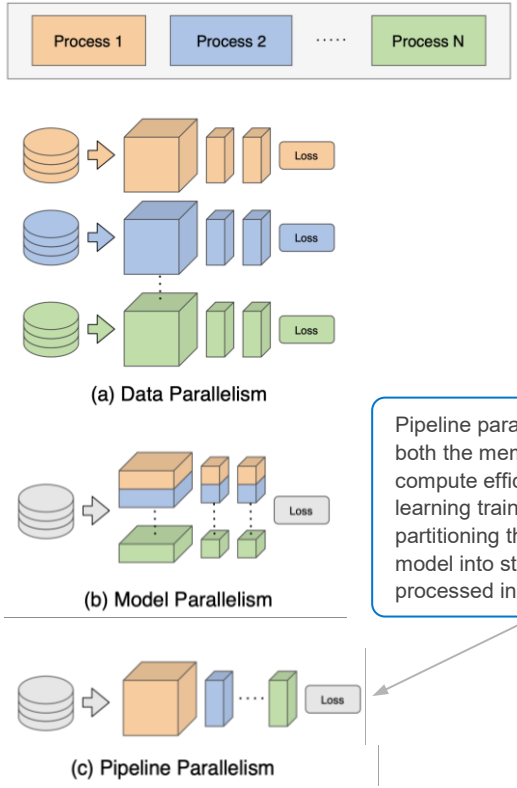
https://www.aivo.co/blog/chatgpt-training-process-advantages-and-limitations



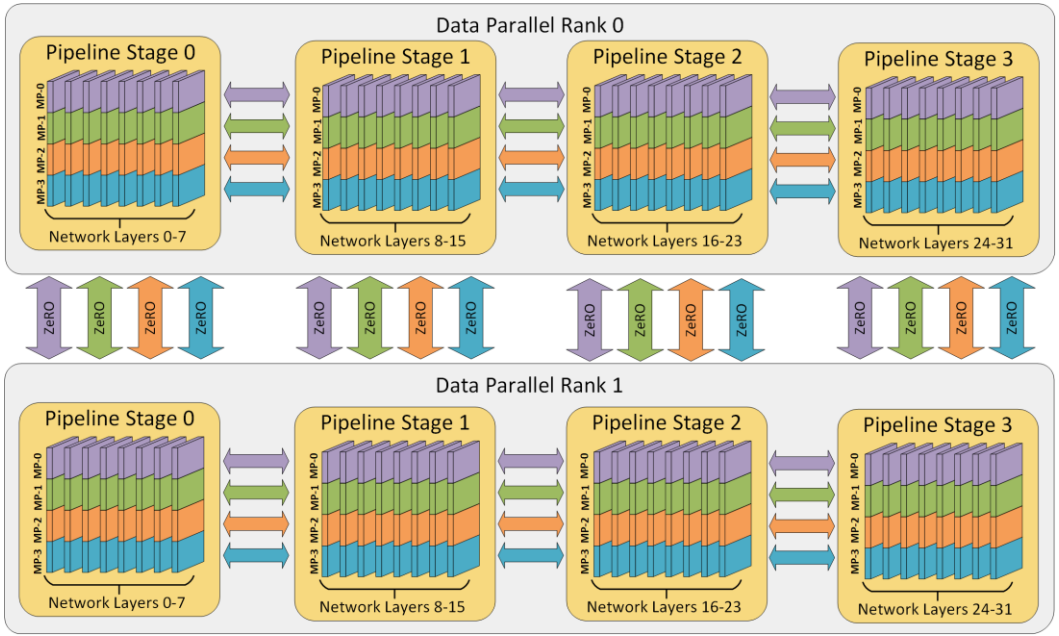
- The output of the RM is used as a scalar reward.
- The supervised policy is fine-tuned to optimize this reward using the PPO algorithm.

https://huggingface.co/blog/rlhf

Data Parallelism, Model Parallelism, Pipeline Parallelism



Pipeline parallelism improves both the memory and compute efficiency of deep learning training by partitioning the layers of a model into stages that can be processed in parallel.

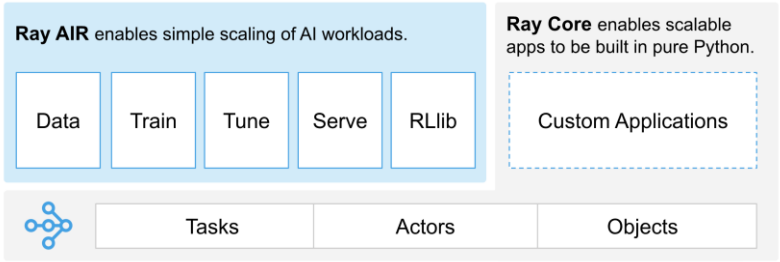


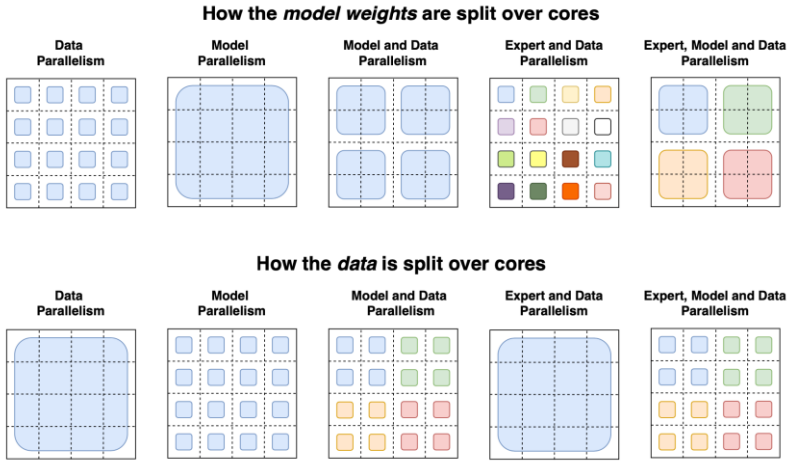
- Each batch of training data is divided into micro-batches that can be processed in parallel by the pipeline stages.
- Once a stage completes the forward pass for a micro-batch, the activation memory is communicated to the next stage in the pipeline.
- Similarly, as the next stage completes its backward pass on a micro-batch, the gradient with respect to the activation is communicated backwards through the pipeline.
- Each backward pass accumulates gradients locally.
- Next, all data parallel groups perform reductions of the gradients in parallel.
- Lastly, the optimizer updates the model weights.

https://www.deepspeed.ai/tutorials/pipeline/

Language Model Training at Scale

- Ray is a unified framework for scaling AI and Python applications. Ray consists of a core distributed runtime and a toolkit of libraries (Ray AIR) for simplifying ML compute.
  - As reported, Ray, the framework from the startup Anyscale, was key in enabling OpenAI to beef up its ability to train ChatGPT at a large scale.
- Alpa is an open-source system for training and serving large-scale neural networks. Alpa aims to automatically parallelizes users' single-device code on distributed clusters with data, operator, and pipeline parallelism.
- One blog post presents how two open-source frameworks, Alpa and Ray, closely integrate to achieve the scale to train a 175B parameters OPT-175B model with pipeline parallelism up to 1024 A100 GPUs in collaboration with Nvidia.
  - Alpa can scale beyond 1000 GPUs for 175 billion parameter scale LLMs.
  - Alpa can achieve peak GPU utilization of 57.5% and HW FLOPs per GPU (179 TFLOPs), about 21%~42% higher compared with published LLM benchmarks from Meta, Google, and Nvidia in 2022.
  - All LLM parallelization and partitioning are done automatically with one line decorator.





https://venturebeat.com/ai/ray-the-machine-learning-tech-behind-openai-levels-up-to-ray-2-0/
https://www.anyscale.com/blog/training-175b-parameter-language-models-at-1000-gpu-scale-with-alpa-and-ray

https://lilianweng.github.io/posts/2021-09-25-train-large/

Stabilizing the Training of LLMs

- During the pre-training of LLMs, it often suffers from the training instability issue, which may cause the model collapse. To address this issue, weight decay and gradient clipping have been widely utilized, where existing studies commonly set the threshold of gradient clipping to 1.0 and weight decay rate to 0.1.
- However, with the scaling of LLMs, the training loss spike is also more likely to occur, leading to unstable training. To mitigate this problem, PaLM and OPT use a simple strategy that restarts the training process from an earlier checkpoint before the occurrence of the spike and skips over the data that may have caused the problem.
- Further, GLM finds that the abnormal gradients of the embedding layer usually lead to spikes, and proposes to shrink the embedding layer gradients to alleviate it.

Model	Batch Size (#tokens)	Learning Rate	Warmup	Decay Method	Optimizer	Precision Type	Weight Decay	Grad Clip	Dropout
GPT3 (175B)	32K→3.2M	6×10^{-5}	yes	cosine decay to 10%	Adam	FP16	0.1	1.0	-
PanGu-α (200B)	-	2×10^{-5}	-	-	Adam	-	0.1	-	-
OPT (175B)	2M	1.2×10^{-4}	yes	manual decay	AdamW	FP16	0.1	-	0.1
PaLM (540B)	1M→4M	1×10^{-2}	no	inverse square root	Adafactor	BF16	lr^2	1.0	0.1
BLOOM (176B)	4M	6×10^{-5}	yes	cosine decay to 10%	Adam	BF16	0.1	1.0	0.0
MT-NLG (530B)	64 K→3.75M	5×10^{-5}	yes	cosine decay to 10%	Adam	BF16	0.1	1.0	-
Gopher (280B)	3M→6M	4×10^{-5}	yes	cosine decay to 10%	Adam	BF16	-	1.0	-
Chinchilla (70B)	1.5M→3M	1×10^{-4}	yes	cosine decay to 10%	AdamW	BF16	-	-	-
Galactica (120B)	2M	7×10^{-6}	yes	linear decay to 10%	AdamW	-	0.1	1.0	0.1
LaMDA (137B)	256K	-	-	-	-	BF16	-	-	-
Jurassic-1 (178B)	32 K→3.2M	6×10^{-5}	yes	-	-	-	-	-	-
LLaMA (65B)	4M	1.5×10^{-4}	yes	cosine decay to 10%	AdamW	-	0.1	1.0	-
GLM (130B)	0.4M→8.25M	8×10^{-5}	yes	cosine decay to 10%	AdamW	FP16	0.1	1.0	0.1
T5 (11B)	64K	1×10^{-2}	no	inverse square root	AdaFactor	-	-	-	0.1
ERNIE 3.0 Titan (260B)	-	1×10^{-4}	-	-	Adam	FP16	0.1	1.0	-
PanGu-Σ (1.085T)	0.5M	2×10^{-5}	yes	-	Adam	FP16	-	-	-

Zhao, Wayne Xin, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, et al. 2023. “A Survey of Large Language Models.” arXiv. https://doi.org/10.48550/arXiv.2303.18223.

Statistics of Large Language Models

	Model	Release Time	Size (B)	Base Model	Adaptation IT	Adaptation RLHF	Pre-train Data Scale	Latest Data Timestamp	Hardware (GPUs / TPUs)	Training Time	Evaluation ICL	Evaluation CoT
Open Source	T5 [71]	Oct-2019	11	-	-	-	1T tokens	Apr-2019	1024 TPU v3	-	✓	-
	mT5 [72]	Mar-2021	13	-	-	-	1T tokens	Apr-2019	-	-	✓	-
	PanGu-α [73]	May-2021	13*	-	-	-	1.1TB	-	2048 Ascend 910	-	✓	-
	CPM-2 [74]	May-2021	198	-	-	-	2.6TB	-	-	-	-	-
	T0 [28]	Oct-2021	11	T5	✓	-	-	-	512 TPU v3	27 h	✓	-
	GPT-NeoX-20B [75]	Feb-2022	20	-	-	-	825GB	Dec-2022	96 40G A100	-	✓	-
	CodeGen [76]	Mar-2022	16	-	-	-	577B tokens	-	-	-	✓	-
	Tk-Instruct [77]	Apr-2022	11	T5	✓	-	-	-	256 TPU v3	4 h	✓	-
	UL2 [78]	Apr-2022	20	-	✓	-	1T tokens	Apr-2019	512 TPU v4	-	✓	✓
	OPT [79]	May-2022	175	-	-	-	180B tokens	-	992 80G A100	-	✓	-
	BLOOM [66]	Jul-2022	176	-	-	-	366B	-	384 80G A100	105 d	✓	-
	GLM [80]	Aug-2022	130	-	-	-	400B tokens	-	768 40G A100	60 d	✓	-
	Flan-T5 [81]	Oct-2022	11	T5	✓	-	-	-	-	-	✓	✓
	mT0 [82]	Nov-2022	13	mT5	✓	-	-	-	-	-	✓	-
	Galactica [35]	Nov-2022	120	-	-	-	106B tokens	-	-	-	✓	✓
	BLOOMZ [82]	Nov-2022	176	BLOOM	✓	-	-	-	-	-	✓	-
	OPT-IML [83]	Dec-2022	175	OPT	✓	-	-	-	128 40G A100	-	✓	✓
	LLaMA [57]	Feb-2023	65	-	-	-	1.4T tokens	-	2048 80G A100	21 d	✓	-
Closed Source	GShard [84]	Jan-2020	600	-	-	-	1T tokens	-	2048 TPU v3	4 d	-	-
	GPT-3 [55]	May-2020	175	-	-	-	300B tokens	-	-	-	✓	-
	LaMDA [85]	May-2021	137	-	-	-	2.81T tokens	-	1024 TPU v3	57.7 d	-	-
	HyperCLOVA [86]	Jun-2021	82	-	-	-	300B tokens	-	1024 A100	13.4 d	✓	-
	Codex [87]	Jul-2021	12	GPT-3	-	-	100B tokens	May-2020	-	-	✓	-
	ERNIE 3.0 [88]	Jul-2021	10	-	-	-	375B tokens	-	384 V100	-	✓	-
	Jurassic-1 [89]	Aug-2021	178	-	-	-	300B tokens	-	800 GPU	-	✓	-
	FLAN [62]	Oct-2021	137	LaMDA	✓	-	-	-	128 TPU v3	60 h	✓	-
	MT-NLG [90]	Oct-2021	530	-	-	-	270B tokens	-	4480 80G A100	-	✓	-
	Yuan 1.0 [91]	Oct-2021	245	-	-	-	180B tokens	-	2128 GPU	-	✓	-
	WebGPT [70]	Dec-2021	175	GPT-3	-	✓	-	-	-	-	✓	-
	Gopher [59]	Dec-2021	280	-	-	-	300B tokens	-	4096 TPU v3	920 h	✓	-
	ERNIE 3.0 Titan [92]	Dec-2021	260	-	-	-	300B tokens	-	2048 V100	28 d	✓	-
	GLaM [93]	Dec-2021	1200	-	-	-	280B tokens	-	1024 TPU v4	574 h	✓	-
	InstructGPT [61]	Jan-2022	175	GPT-3	✓	✓	-	-	-	-	✓	-
	AlphaCode [94]	Feb-2022	41	-	-	-	967B tokens	Jul-2021	-	-	-	-
	Chinchilla [34]	Mar-2022	70	-	-	-	1.4T tokens	-	-	-	✓	-
	PaLM [56]	Apr-2022	540	-	-	-	780B tokens	-	6144 TPU v4	-	✓	✓
	AlexaTM [95]	Aug-2022	20	-	-	-	1.3T tokens	-	128 A100	120 d	✓	✓
	Sparrow [96]	Sep-2022	70	-	-	✓	-	-	64 TPU v3	-	✓	-
	U-PaLM [97]	Oct-2022	540	PaLM	-	-	-	-	512 TPU v4	5 d	✓	✓
	Flan-PaLM [81]	Oct-2022	540	PaLM	✓	-	-	-	512 TPU v4	37 h	✓	✓
	Flan-U-PaLM [81]	Oct-2022	540	U-PaLM	✓	-	-	-	-	-	✓	✓
	GPT-4 [46]	Mar-2023	-	-	✓	✓	-	-	-	-	✓	✓
	PanGu-Σ [98]	Mar-2023	1085	PanGu-α	-	-	329B tokens	-	512 Ascend 910	100 d	✓	-

Training a 540-Billion Parameter Language Model with Pathways

- PaLM demonstrates the large-scale use of the Pathways system to scale training to 6144 chips, the largest TPU-based system configuration used for training to date. The training is scaled using data parallelism at the Pod level across two Cloud TPU v4 Pods, while using standard data and model parallelism within each Pod.
- PaLM achieves a training efficiency of 57.8% hardware FLOPs utilization, the highest yet achieved for LLMs at this scale.

https://ai.googleblog.com/2022/04/pathways-language-model-palm-scaling-to.html

Zhao, Wayne Xin, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, et al. 2023. “A Survey of Large Language Models.” arXiv. https://doi.org/10.48550/arXiv.2303.18223.

Stories in Opt-175B Training Run

- Opt-175B model's training run on 300B tokens required ~4.30E+23 FLOPs of compute, or roughly ~33 days of continuous training on 1024 80GB A100s (assuming no hardware issues, no numerical instabilities, etc.).

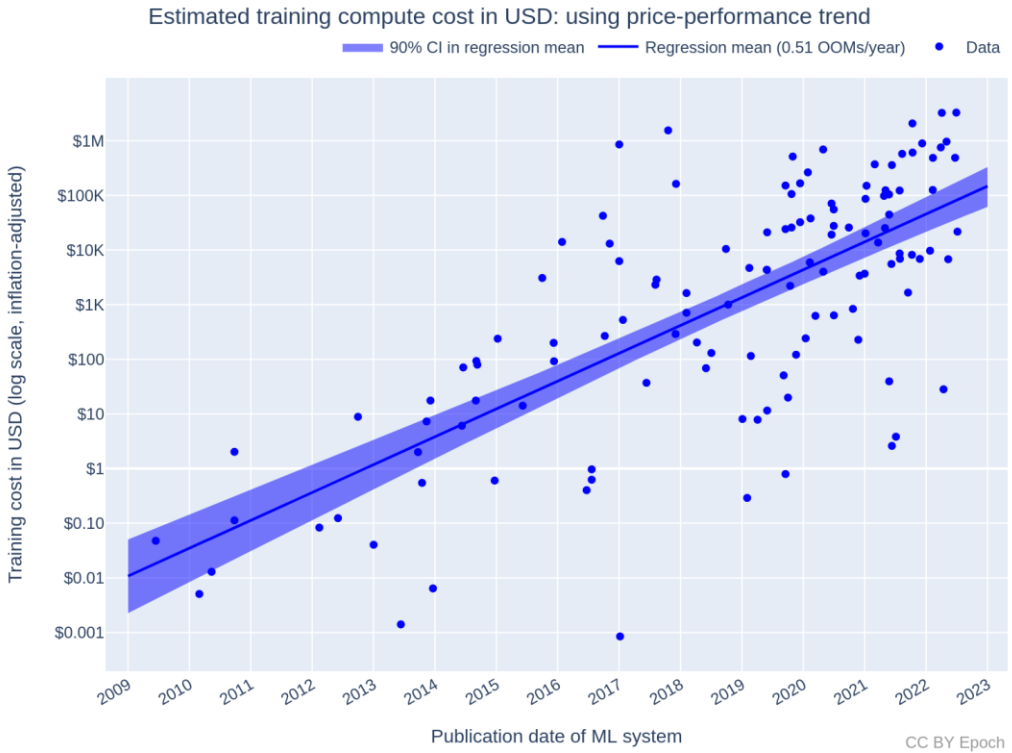
- LLM development at scale is an extraordinarily resource-intensive process. Ensuring training converges at this scale is also highly nontrivial without sufficient ablations at "medium" scale. Results obtained from training at "small" scale (< ~13B params) also do not necessarily hold when "scaled-up".

- The 148 pages of notes cover ~90 restarts over the course of training the lineage of this current model (experiments 12.xx).

- The team started gluing together all of our monitoring and health-checks for the cluster in order to enable fully automated recovery. It took several iterations to get right, but we were able to automatically recover from 8 hardware failures between Christmas and New Years, greatly reducing the burden of the on-call.

https://github.com/facebookresearch/metaseq/blob/main/projects/OPT/chronicles/final_update.md

Training Cost in Dollars



https://epochai.org/blog/trends-in-the-dollar-training-cost-of-machine-learning-systems

- Figure 1: estimated cost of compute in US dollars for the final training run of ML systems. The costs here are estimated based on the trend in price-performance for all GPUs in Hobbhahn & Besiroglu (2022).
- It combines training compute and GPU price-performance data to estimate the cost of compute in US dollars for the final training run of 124 machine learning systems published between 2009 and 2022, and find that the cost has grown by approximately 0.5 orders of magnitude per year.
- The cost estimates have large uncertainty bounds—the true costs could be several times larger or smaller. The cost estimates are themselves built on top of estimates (e.g. training compute estimates, GPU price-performance estimates, etc.).
- The cost estimates only cover the compute for the final training runs of ML systems—nothing more.

Transformative Force for Business Change

Back to Table of Contents

What a Large Language Model Used For

- A Large Language Model (LLM) can be used to assimilate vast amounts of knowledge from massive datasets, enabling it to recognize, summarize, translate, predict and generate text and other content.

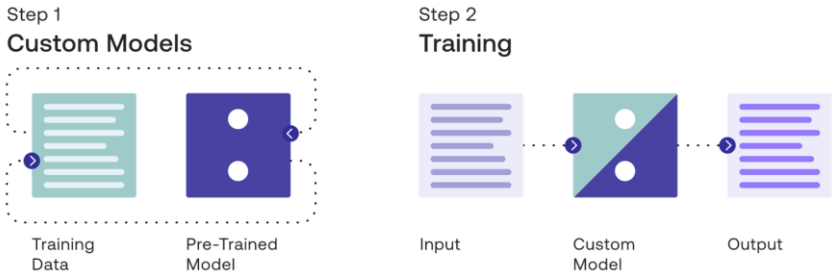
- It may be that LLMs will assist us in highly creative tasks by seeding us with initial ideas that we can build upon and refine.
  - These tools are not going to get everything right but they will help solve that initial writer's block.
  - When you're staring at a blank page, you can describe the prompt or problem and the model outputs something, it may give you a good starting point and show you something you can use — even if it's not entirely what you want.
  - Prompt engineering (i.e. instructing models using the right starting text) will become a new way of programing, using natural language.

- LLM is also an automation tool that elevates user experience and work efficiency.
  - Both simple and more complex daily tasks can be automated by LLMs, freeing up many hours on a practitioner's weekly schedule and reducing workload.

Rise of ChatGPT is Driving a Paradigm Shift in NLP

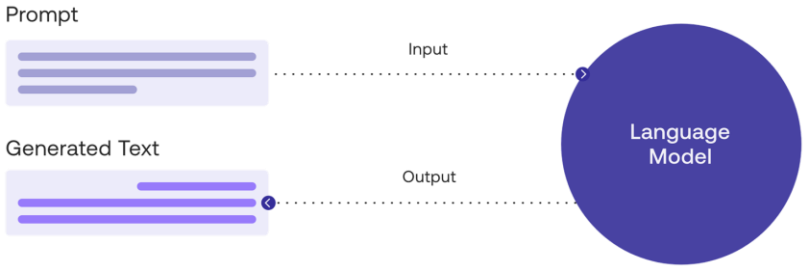
- Since its public launch in November 2022 by OpenAI, ChatGPT has captured the world's attention.
- OpenAI set a new standard in terms of how we can interact with AI models in a comfortable and natural way like answering follow-up questions, questioning false premises, rejecting inappropriate requests, and even admitting mistakes.
- The focus in the "Pre-train, Fine-tune" paradigm has been on the question: "How should we fine-tune language models?". However, the rise of ChatGPT essentially shifts the focus into: "How should we design the input to language models?"

The previous "Pre-train, Fine-tune" paradigm in NLP has been to use pre-trained language models and then fine-tune them with additional training data for downstream tasks.



https://docs.cohere.ai/docs/training-custom-models

By clever design of the prompt templates, i.e. prompt engineering or prompt design, we can adapt pre-trained language models to perform new tasks without any extensive fine-tuning.



https://docs.cohere.ai/docs/prompt-engineering

Picking the Right Model for Downstream Tasks

- Language modelling is a powerful upstream task, NLP is mostly used for more targeted downstream tasks such as sentiment analysis, question answering and information extraction.
  - Autoregressive models perform well on text generation tasks such as conversational AI, question answering and text summarization, while auto-encoders excel at "understanding" and structuring language, for example for sentiment analysis and various information extraction tasks.
  - Models intended for zero-shot learning can theoretically perform all kinds of tasks as long as they receive appropriate prompts — however, their accuracy is generally lower than that of fine-tuned models.

Model	Core differentiator	Pre-training objective	Para-meters	Access	Information Extraction	Text Classification	Conversa-tional AI	Summari-zation	Machine Translation	Content generation
BERT	First transformer-based LLM	AE	370M	Source code						
RoBERTa	More robust training procedure	AE	354M	Source code						
GPT-3	Parameter size	AR	175B	API						
BART	Novel combination of pre-training objectives	AR and AE	147M	Source code						
GPT-2	Parameter size	AR	1.5B	Source code						
T5	Multi-task transfer learning	AR	11B	Source code						
LaMDA	Dialogue; safety and factual grounding	AR	137B	No access						
XLNet	Joint AE and AR	AE and AR	110M	Source code						
DistilBERT	Reduced model size via knowledge distillation	AE	82M	Source code						
ELECTRA	Computational efficiency	AE	335M	Source code						
PaLM	Training infrastructure	AR	540B	No access						
MT-NLG	Training infrastructure	AR and AE	530B	API						
UniLM	Optimised both for NLU and NLG	Seq2seq, AE and AR	340M	Source code						
BLOOM	Multilingual (46 languages)	AR	176B	Source code						

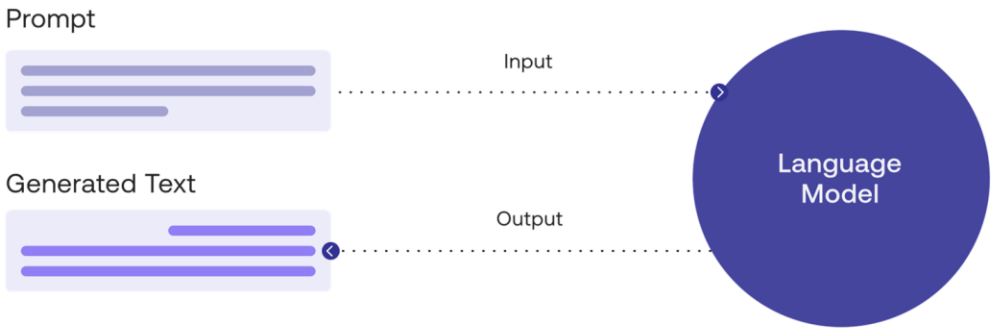
AR = Autoregression
AE = Autoencoding
Seq2seq = Sequence-to-sequence

Highly appropriate
Appropriate
Somewhat appropriate

https://towardsdatascience.com/choosing-the-right-language-model-for-your-nlp-use-case-1288ef3c4929

Prompt Engineering

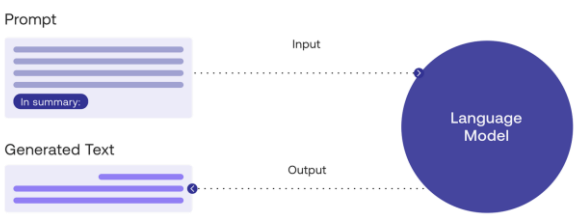
- Prompt engineering is a relatively new discipline for developing and optimizing prompts to efficiently use language models (LMs) for a wide variety of applications and research topics.
- Prompt engineering skills help to better understand the capabilities and limitations of large language models.
- There are certain elements that make up a prompt. A prompt can contain any of the following components:
  - **Instruction** - a specific task or instruction you want the model to perform
  - **Context** - can involve external information or additional context that can steer the model to better responses
  - **Input Data** - is the input or question that we are interested to find a response for
  - **Output Indicator** - indicates the type or format of output.
  - *Not all the components are required for a prompt and the format depends on the task at hand.*



https://github.com/dair-ai/Prompt-Engineering-Guide

Principles of Designing Prompts

A prompt guides the model to generate useful output

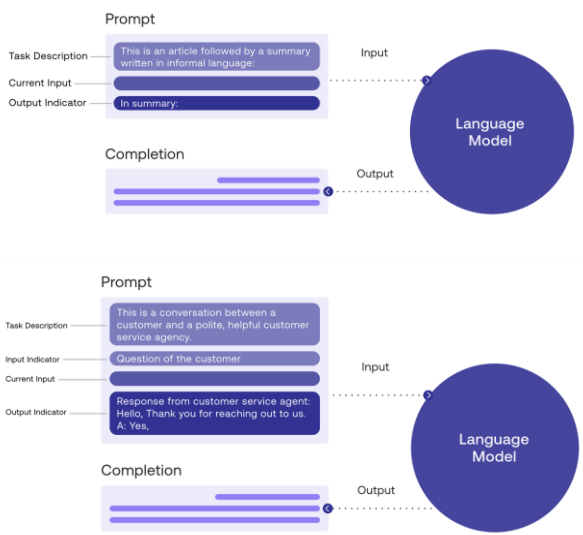


This prompt has two components: the text you want summarized, and the task description.

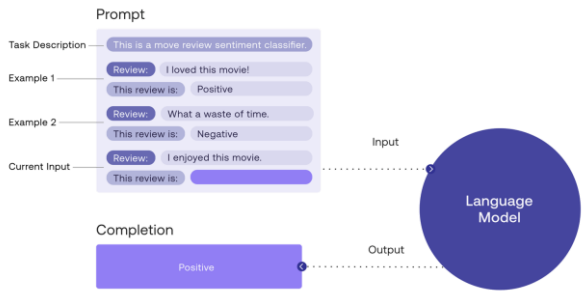
Try multiple formulations of the prompt to get the best generations

- When using generate, it is useful to try a range of different prompts for the problem you are trying to solve. Different formulations of the same prompt which might sound similar to humans can lead to generations that are quite different from each other.

Describe the task and the general setting



Show the model what you would like to see



- As you get started with designing prompts, you should keep in mind that it is really an iterative process that requires lot of experimentation to get optimal results.
- You can start with simple prompts and keep adding more elements and context as you aim for better results

https://docs.cohere.ai/docs/prompt-engineering

Questioning the Model's Answers

- Although we tend to perceive generative AI through a human lens, its mistakes are not the ones in a form of human. It makes the mistakes of a different form of intelligence based on pattern recognition. We should not identify these mistakes as errors.
  - Can we create an interrogative mode that can question the precision of a model's responses, even in situations where we are not aware of the answers beforehand?

- Cognitive limitations may keep humans from uncovering truths buried in massive information. At this point, we need computers or AI models to harness growing volumes of data.
  1) Learn to use the models to get answers
  2) Question the accuracy of a model's answers, assess whether and to what degree its answers are worthy of confidence
     - Even if generative AI models become fully interpretable and accurate, they will still present inherent challenges to human values and ethics.
  3) Humans develop the confidence and ability to challenge the outputs of AI systems

- Therefore, it implies a collaboration not only with a distinct type of technical entity but also with a unique mode of reasoning, and this dependency is likely to trigger a transformation in metacognition — the understanding of understanding (*Kissinger, Schmidt, and Huttenlocher 2023*).